

Appunti su Basi di Dati

Riccardo Di Michele

1 introduzione

Nell'ambito informatico abbiamo sviluppato un bisogno di organizzare le *informazioni* che ci interessano, per poterle semplicemente conservare o *lavorarci*. Il modo in cui queste informazioni vengono rappresentate è tramite i **dati**, quest'ultimi devono però seguire una certa **codifica**; vediamo un esempio.



I dati che noi estraiamo da questi cartelli (le nostre informazioni) sono gli stessi: " *divieto dalle 8 alle 17* ". Ma in questo caso la codifica dei dati è molto importante poichè il modo in cui l'orario viene scritto ci darà *altretanti* dati. Infatti l'orario all'interno delle parentesi indica che riguarda solo il sabato, quello indicato in rosso è solo nei giorni festivi, senza nessun'altra formattazione invece tutti gli altri giorni.

Per ogni dato dovremmo quindi *specificare* come viene codificato, o semplicemente, come verrà interpretato, questo può essere un lavoro molto faticoso quando andremo a lavorare su migliaia e milioni di dati. Queste **basi di dati** verranno allora gestite da un **DBMS**, **DataBase Management System**. Un **DBMS** ci permette di gestire con facilità dati di **grandi** dimensioni (anche 500 Terabyte) e permette di condividere quest'ultimi attraverso più utenti (come tra i vari settori di una azienda); quest'ultima qualità potrebbe

però portare a qualche **conflitto**, se per esempio dipendente A vuole accedere allo stesso dato su cui sta lavorando dipendente B potrebbero formarsi dei dati non accurati.

Per questo motivo i **DBMS** offrono anche meccanismi di autorizzazione, quindi è possibile definire quale utente è autorizzato a leggere o modificare una certa raccolta di dati. Un aspetto delicato che deve gestire un **DBMS** è quindi quello delle **transizioni**, ovvero tutte quelle operazioni sui dati che devono essere **atomiche**, ovvero operazioni non scindibili, devono essere portate a termine per forza di cose, anche nel caso di transizioni concorrenti (come spiegato prima, quando due utenti operano sugli stessi dati).

2 Descrizione dei dati

Il **modello dei dati** è l'insieme delle tecniche e dei costrutti usati per organizzare i dati e descriverne la loro natura.

Per esempio il **modello relazionale** ci permette di costruire **record** omogenei; vediamo un esempio.

Orario

Insegnamento	Docente	Aula
Analisi I	Luigi Neri	N1
Chimica	Piero Rossi	N2
Basi di Dati	Angela Valenti	N3
Fisica II	Vittorio Strano	N4

La *relazione* di cui parla il modello non è altro che l'intera tabella Orario, ed è composta da righe omogenee, ovvero righe che hanno la stessa struttura: un **insegnamento**, un **docente** e un **aula**.

Sempre considerando la tabella possiamo adesso dare la definizione di **tupla** o (**record**) e **attributo**: la prima non è altro che una riga della tabella e la seconda è invece una colonna.

In ogni base di dati possiamo individuare uno **schema**, ovvero una *descrizione* invariante nel tempo che descrive la struttura della relazione; nella relazione *Orario* vista prima lo schema è l'insieme di tutti gli attributi (ovvero le colonne della relazione).

Troveremo inoltre l'**istanza**, che è invece variabile nel tempo, ovvero i valori della tabella stessa; la colonna *Insegnamento* è per esempio una istanza.

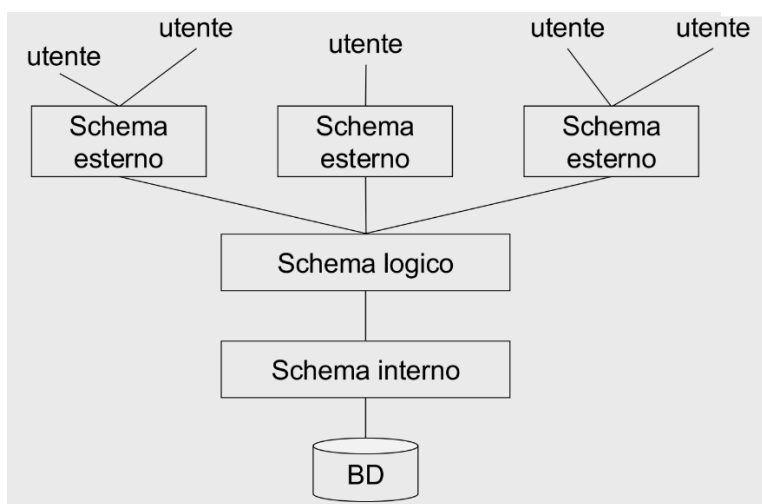
Abbiamo già parlato del *modello relazionale*, ma questo tipo fa in realtà

parte di una macro-categoria di modelli chiamati **modelli logici**, che descrivono i dati, e le relazioni tra loro, usando strutture "comprensibili" ad un software (per esempio tramite **XML**).

Un'altra categoria è quella dei **modelli concettuali** che invece mirano soltanto a rappresentare i dati a livello grafico, usando per esempio il famoso schema **E-R**, *Entità-Relazione*.

TODO METTI UNA FOTO DEL MODELLO ER

Di seguito vedremo una architettura molto comune per i DBMS, chiamata architettura **ANSI/SPARC**:



Ogni schema dell'architettura ha una sua funzione e soprattutto lavora soltanto su una parte del database.

Lo **schema esterno** è quello a cui si interfaccierà l'utente, di conseguenza sarà molto "semplificato". Se per esempio prendiamo il sito di una università l'utente medio sarà uno studente, lo schema esterno lavorerà quindi (e farà vedere) soltanto sui voti di quello studente; il database dell'università ha ovviamente altri dati (nomi dei professori, eventi, gli esami degli scorsi anni), ma farà solo vedere allo studente ciò che a lui davvero serve.

Al cuore della struttura troviamo invece lo **schema logico** che descrive in modo concettuale quali sono i dati e le relazioni tra quest'ultimi (per esempio tramite il *modello ER*, come detto prima)

L'ultimo è lo **schema interno** che descrive come sono *fisicamente* memorizzati questi dati e su quali dispositivi (*HDD*, *SDD* etc)

Il punto forte di questa struttura è l'**indipendenza dei dati**, infatti potremmo cambiare il modo in cui "descriviamo i dati" (schema logico), senza cambiare poi gli altri schemi o addirittura il database stesso.

3 Interrogare il database

Abbiamo parlato di come descriviamo e come conserviamo questi famigerati *dati*, ma come lavoriamo su quest'ultimi?

Useremo un linguaggio interattivo: **SQL**. Quest'ultimo è un linguaggio standard oramai per interfacciarsi con database e lo troveremo già incluso in tanti linguaggi come **C** oppure **Pascal**.

Il linguaggio è composto principalmente da due tipologie di operazioni: **DML** e **DDL**.

Le operazioni di **Data Manipulation Language** manipolano le istanze del database, e quindi modificano i dati al suo interno; il secondo tipo di operazioni, **Data Definition Language**, definiscono lo schema del database e come i dati si relazionino tra di loro.

Seguono degli esempi di, rispettivamente, operazione di DDL e DML

```
CREATE TABLE orario (  
    insegnamento CHAR(20),  
    docente CHAR(20),  
    aula CHAR(4),  
    ora CHAR(5)  
);  
  
INSERT INTO orario (insegnamento, docente, aula, ora)  
VALUES ('Basi di dati', 'Mario Rossi', 'N1', '09:00');
```

Questi esempi servono soltanto per avere un rapido chiarimento sulle differenze tra i due tipi di comandi, ovviamente una persona che non ha mai usato SQL avrà difficoltà a capire. Basta sapere che il primo comando **CREATE TABLE** definisce lo schema della tabella *orario*, il secondo comando (**INSERT INTO**), va invece a modificare questa tabella aggiungendo dei nuovi dati.

4 Personaggi del database

Infine vedremo le principali figure che si intersecano e lavorano col database. Il primo tra tutti è il **Database Administrator (DBA)**, che definisce lo

schema della base di dati, gestisce la sicurezza del database (quindi decide chi è autorizzato ad accedere a cosa), si occupa della manutenzione, delle prestazioni e dei backup del database e, infine, coordina gestisce le esigenze dei vari utenti; per esempio se una azienda A vuole accedere agli stessi dati di una azienda B il **DBA** progetterà la base di dati in modo tale che entrambe le aziende siano soddisfatte. Abbiamo inoltre la figura del **programmatore**, che si occupa di scrivere applicazioni che interfaccino l'utente col database, in base al tipo di utente ci saranno ovviamente applicativi diversi; per esempio un utente comune non avrà sicuramente accesso a ogni dato del database di una banca.

In modo simile gli **analisti** invece sono utenti ma con conoscenze del linguaggio SQL, quindi non usano applicativi ma sono loro stessi che "interrogano" il database con query scritte da loro.

Infine gli **utenti finali** (detti **naviganti**) sono gli utenti più comuni, quelli che interagiscono con le applicazioni scritte dai *programmatori* già citati prima.